

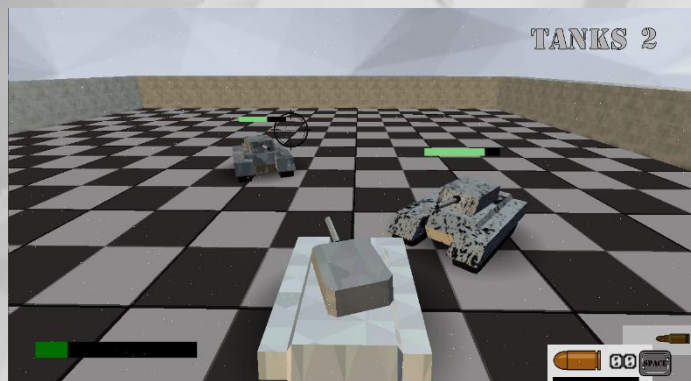
作品説明資料

トライデントコンピュータ専門学校
ゲームサイエンス学科3年 吉川佳汰

作品紹介 (個人制作)

MonochromeTank

MonochromeTank



▶開発ツール

Visual Studio
DirectX11

▶使用言語

C++

▶制作期間

2024年8月～
2025年7月
(2～3年生)

〈コンセプト〉

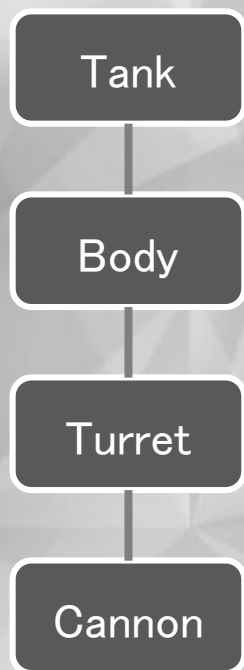
モノクロ世界で戦車同士で戦い、最後の一人まで生き残るバトルロイヤル3Dシューティングアクション

アピールポイント

MonochromeTank

Compositeパターンを用いた戦車の設計

戦車のパーツを親パーツに付けていく形にすることで移動や回転の処理を各パーツで行う必要がなくなり、コードの**拡張性**と**可読性**が向上した。



```
/**
 * @file IParent.h
 * @brief Parentインターフェース
 */
#pragma once
#include "Framework/Graphics.h"
#include "Framework/Resources.h"
#include "Interface/IParts.h"

class IParent
{
public:
    // デストラクタ
    virtual ~IParent() = default;
    // 初期化处理
    virtual void Initialize() = 0;
    // 更新処理
    virtual void Update(float elapsedTime) = 0;
    // 描画処理
    virtual void Render() = 0;
    // 終了処理
    virtual void Finalize() = 0;

    // 部品の追加
    virtual void Attach(std::unique_ptr<IParts> parts) = 0;
    // 部品の削除
    virtual void Detach(std::unique_ptr<IParts> parts) = 0;
};
```

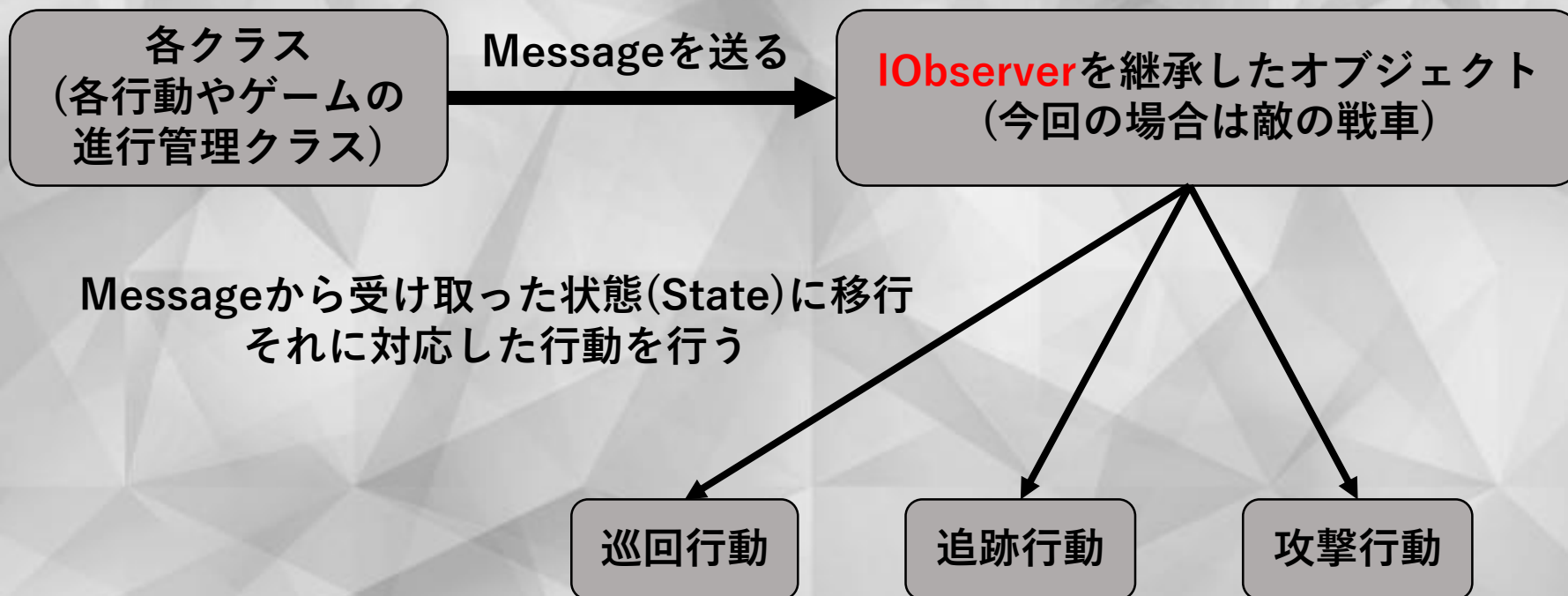
```
/**
 * @file IParts.h
 * @brief Partsインターフェース
 */
#pragma once
#include "Framework/Graphics.h"
#include "Framework/Resources.h"

class IParts
{
public:
    // デストラクタ
    virtual ~IParts() = default;
    // 初期化处理
    virtual void Initialize() = 0;
    // 更新処理
    virtual void Update(
        float elapsedTime,
        const DirectX::SimpleMath::Vector3& currentPosition,
        const DirectX::SimpleMath::Quaternion& currentRotation
    ) = 0;
    // 描画処理
    virtual void Render() = 0;
    // 終了処理
    virtual void Finalize() = 0;

    // 部品の追加
    virtual void Attach(std::unique_ptr<IParts> parts) = 0;
    // 部品の削除
    virtual void Detach(std::unique_ptr<IParts> parts) = 0;
};
```

State、Observerパターンを用いた敵の行動遷移

敵の行動はStateパターンで管理することでクラスの分割が可能になり、**拡張性**と**可読性**が向上した。また行動遷移にObserverパターンを利用して他のクラスから簡単に行動変更の通知を送ることが可能になった。

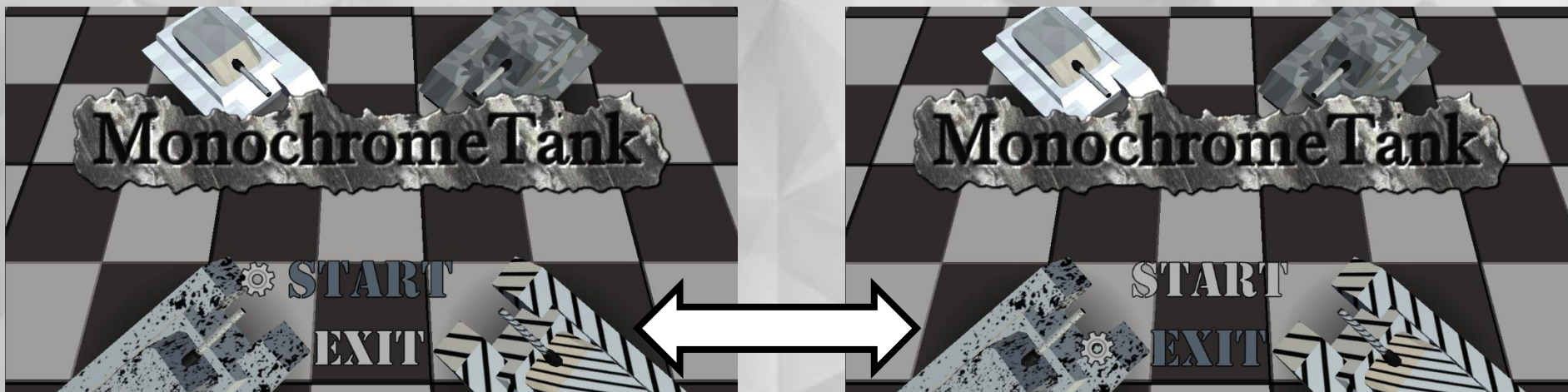


アピールポイント

MonochromeTank

ButtonのUIの作成

マウスクリックで処理を行うことが出来るようにButtonクラスを作成した。テクスチャ、スケール、座標情報からボタン用の矩形を計算、std::functionを用いてマウスクリック、オーバー時の処理を渡しておくことで複数のボタンの処理を一括で行うことが可能になった。

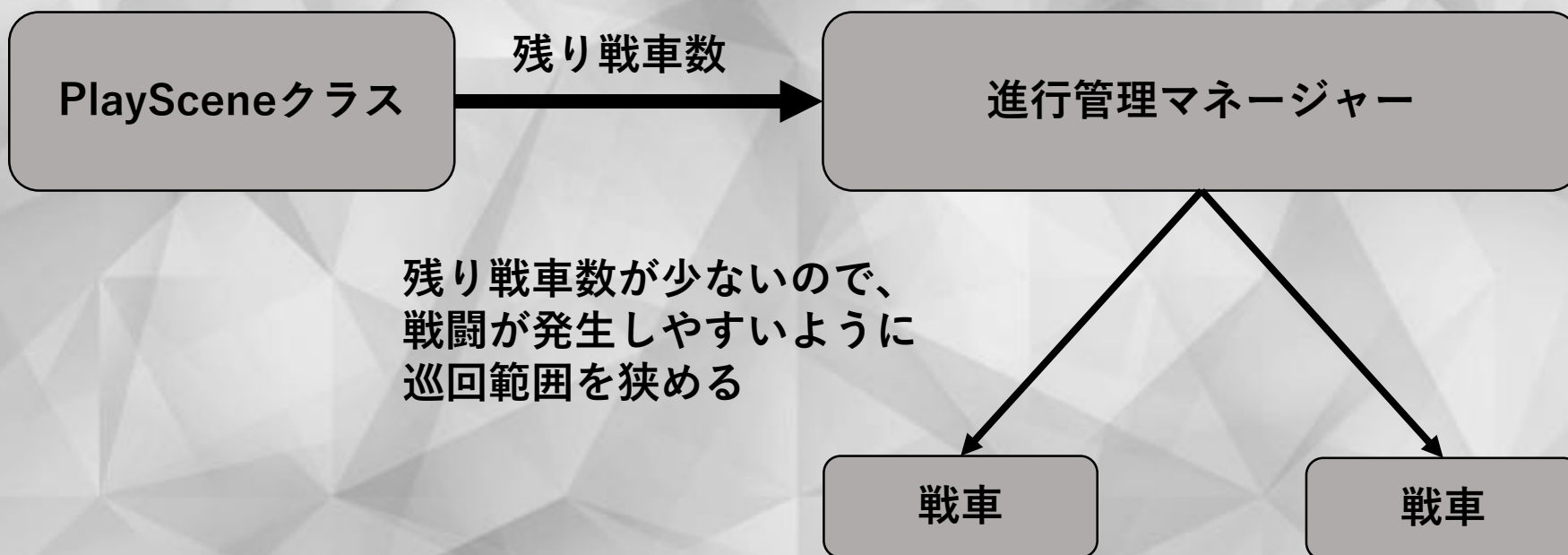


マウスオーバーで選択しているボタンを変更可能に！

全体の進行を管理するメタAIのクラスの実装

全ての戦車の情報を持ち、戦車の現在の行動やPlaySceneから受け取ったゲームの進行度に応じて各戦車に命令を与えるクラスを作成した。

〈例〉

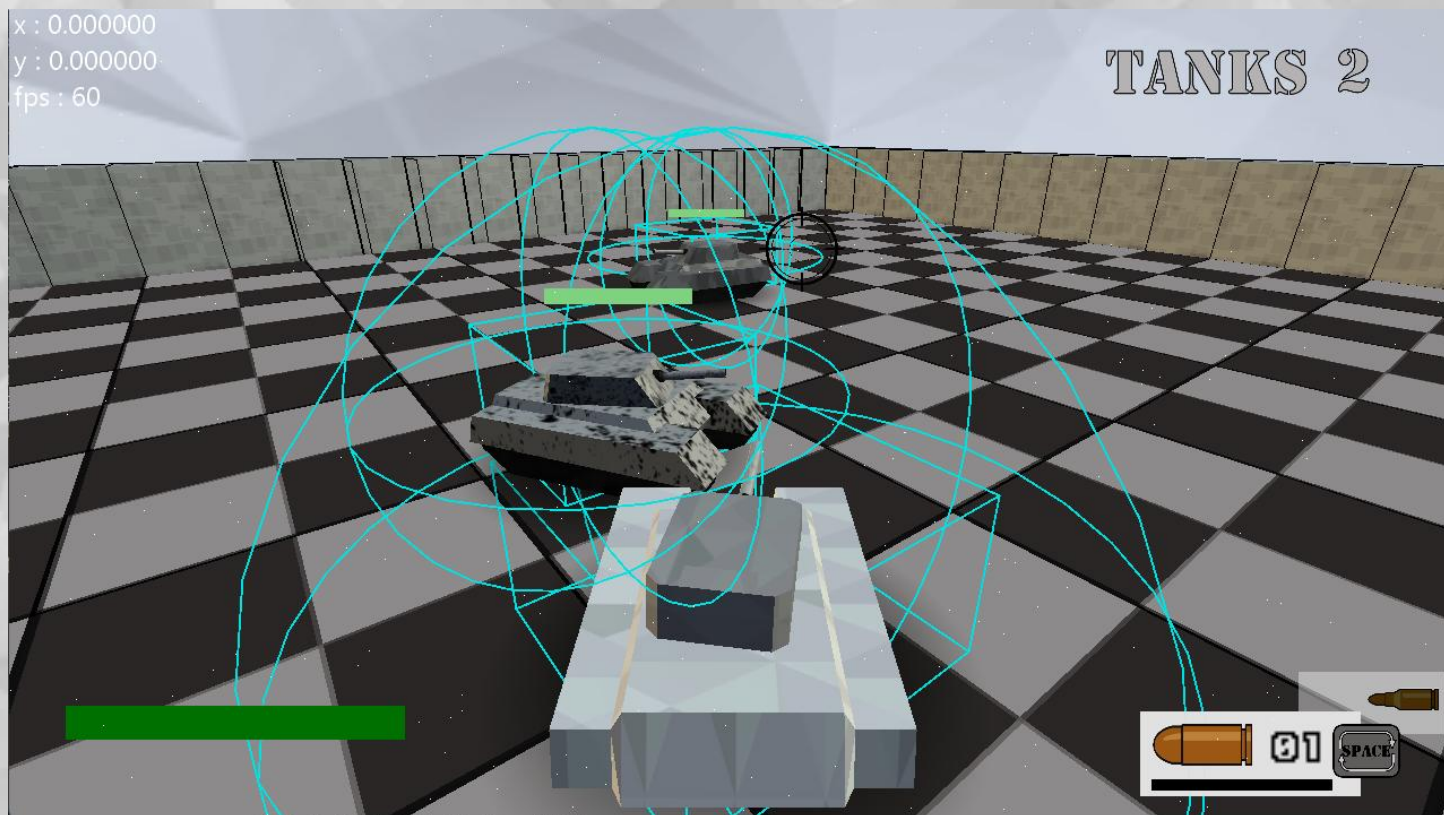


アピールポイント

MonochromeTank

デバッグ表示による開発の効率化

各戦車のコライダー等の情報をデバッグビルドでのみ表示を行うことでデバッグ作業を効率化した。



CollisionManagerクラスの作成

全てのオブジェクトのコライダー情報を持つクラスを作成することで当たり判定の処理を各オブジェクトで行う必要がなくなりコードの**可読性**が向上した。

```
class CollisionManager
{
public:
    CollisionManager();
    ~CollisionManager() = default;
    // 更新処理
    void Update();

public:
    // オブジェクトの設定
    void SetObjectData(
        std::vector<Tank*> tanks,
        mylib::FollowCamera* camera,
        FixedTurret* fixedTurret,
        std::vector<Wall*> walls,
        WallGimmick* wallGimmick
    )
    {
        m_tanks = tanks;
        m_camera = camera;
        m_fixedTurret = fixedTurret;
        m_walls = walls;
        m_wallGimmick = wallGimmick;
    }
};
```

```
private:
    // 各オブジェクト
    std::vector<Tank*> m_tanks;           // 戦車
    mylib::FollowCamera* m_camera;       // カメラ
    FixedTurret* m_fixedTurret;          // 固定砲台
    std::vector<Wall*> m_walls;           // 壁
    WallGimmick* m_wallGimmick;          // 壁ギミック

private:
    // 戦車と通常弾の当たり判定
    void DetectCollisionTankAndNormalBullets();
    // 戦車と砲弾の当たり判定
    void DetectCollisionTankAndCannonBall();
    // 戦車同士の当たり判定
    void DetectCollisionTankAndOtherTanks();
    // 戦車と壁の当たり判定
    void DetectCollisionTankAndWalls();
    // 弾と壁の当たり判定
    void DetectCollisionBulletsAndWalls();
};
```

アピールポイント

MonochromeTank

外部ファイルによるステージ、定数の読み込み

ステージは**CSV**形式、パラメータは**JSON**形式で読み込み



- ・ **プログラマー以外**も変更可能！
- ・ **再コンパイルが不要**に！

```
"TURRET_ANGLE_MIN": -45,
"TURRET_ANGLE_MAX": 45,
"CANNON_ANGLE_MIN": -7.5,
"CANNON_ANGLE_MAX": 22.5,

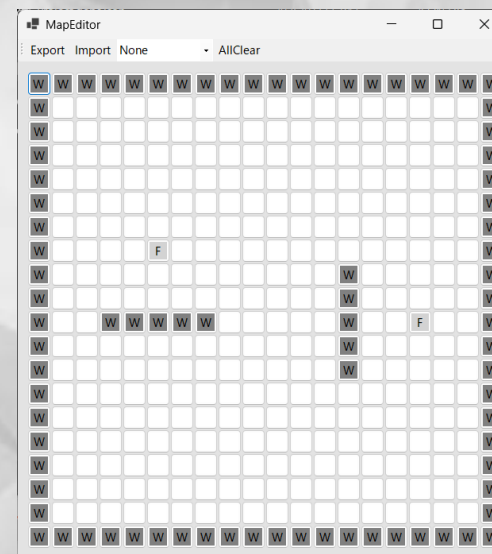
"PLAYER_POSITION": [{"x": -6.0, "y": 0.0, "z": 6.0}],
"PLAYER_SPEED": 3.0,
"PLAYER_ROTATION_SPEED": 0.75,
"PLAYER_HP": 50,

"ENEMY_POSITIONS": [
  [{"x": 6.0, "y": 0.0, "z": -6.0}],
  [{"x": -6.0, "y": 0.0, "z": -6.0}],
  [{"x": 6.0, "y": 0.0, "z": 6.0}]
],
"ENEMY_SPEED": 2.0,
"ENEMY_ROTATION_SPEED": 1.5,
"ENEMY_HP": 50,

"SHOT_INTERVAL": 0.15,

"BULLET_COUNT": 20,
"BULLET_RELOAD_TIME": 1.0,
"BULLET_SURVIVAL_TIME": 1.5,
"BULLET_SPEED": [{"x": 0.0, "y": 0.0, "z": -15.0}],
"BULLET_COLLIDER_RADIUS": 0.05,
"BULLET_DAMAGE": 1,
"BULLET_MAX_TRAIL": 7,
"BULLET_TRAIL_WIDTH": 0.015,

"CANNONBALL_RELOAD_TIME": 2.0,
"CANNONBALL_SURVIVAL_TIME": 5.0,
"CANNONBALL_SPEED": [{"x": 0.0, "y": 0.0, "z": -10.0}],
```



自作のステージエディタ (C#)